

1.概要

关于同步

简易的同步，就是写视频 AVpacket 和音频 AVPacket 的时候，会算出他们的 pts，谁的 pts 更小，就写谁的包到媒体流中。

稳帧

即使使用了定时器，视频帧和音频帧也不能达到每秒钟写入指定帧数数据，比如视频采集，因为定时器的原因，有时候可以达到 1 秒钟 25 帧，有的时候可以达到 24，有时候是 26，这使得播放视频会出现跳帧，不流畅。

为了避免这个问题，可以多采集一些，然后视频帧有对应的时间，一帧画面写完之后，推算出下一帧的时间，然后未达到这个时间的视频帧都丢弃掉，如果，视频帧不够，那么就写入上一帧的数据。

关于优化

为了实现上面说的稳帧，以及让播放更流畅，可以考虑使用**视频队列和音频队列**，就像解码视音频一样。

也就是说，视频队列和音频队列是完成上面同步和稳帧的前提。

2.实现过程

2.1 音频视频队列

首先，采集到的数据，不是直接写入视频流，而是先写入队列，然后申请写线程，写线程中不断从队列里面取出数据，写入视频流。

队列的话，需要添加元素，和取出元素.两个方法

实现如下

```
struct BufferDataNode
{
    uint8_t * buffer;
    int bufferSize;
    int64_t time; //视频帧用于稳帧，比较时间
};
```

添加成员

```
QList<BufferDataNode*> m_videoDataList;
QList<BufferDataNode*> m_audioDataList;
BufferDataNode* lastVideoNode;
```

```

bool m_videoBeginFlag;
QMutex m_videoMutex;
QMutex m_audioMutex;
bool isStop;
int64_t video_pts;
int64_t audio_pts;

void videoDataQuene_Input(uint8_t *buffer, int size, int64_t time);
BufferDataNode *videoDataQuene_get(int64_t time);
void audioDataQuene_Input(const uint8_t *buffer, const int &size);
BufferDataNode *audioDataQuene_get();

void SaveVideoFileThread::videoDataQuene_Input(uint8_t * buffer, int size, int64_t
time)
{
    BufferDataNode * node = (BufferDataNode*)malloc(sizeof(BufferDataNode));
    node->bufferSize = size;
    node->time = time;
    node->buffer = buffer;

    m_videoMutex.lock();
    m_videoDataList.append(node);
    m_videoMutex.unlock();

}

BufferDataNode *SaveVideoFileThread::videoDataQuene_get(int64_t time)
{
    BufferDataNode * node = NULL;

    m_videoMutex.lock();
    while(m_videoDataList.size() != 0 )
    {
        node = m_videoDataList.takeFirst(); //取出元素并在链表移除该点
        if( m_videoDataList.size() == 0) break;
        if( time > node->time ) //淘汰掉小于下一帧时间的数据
        {
            av_free(node->buffer);
            node->buffer = NULL;
            free(node);
            node = NULL;
        }else
        {

```

```

        break;
    }
}
m_videoMutex.unlock();
return node;
}

void SaveVideoFileThread::audioDataQuene_Input(const uint8_t *buffer, const int &size)
{
    BufferDataNode *node = (BufferDataNode*)malloc(sizeof(BufferDataNode));

    node->bufferSize = size;
    node->buffer = (uint8_t*)buffer;

    m_audioMutex.lock();
    m_audioDataList.append(node);
    m_audioMutex.unlock();
}

BufferDataNode * SaveVideoFileThread::audioDataQuene_get()
{
    BufferDataNode *node = NULL;
    m_audioMutex.lock();
    if (m_audioDataList.size() != 0 ) {
        node = m_audioDataList.takeFirst();
    }
    m_audioMutex.unlock();
    return node;
}

```

2.2 开启视频录制

```

void SaveVideoFileThread::slot_openVideo()
{
    if( m_avFormat.hasCamera || m_avFormat.hasDesk) {
        m_videoBeginFlag = true;
        m_picInPicRead->slot_openVideo();
    }

    if( m_avFormat.hasAudio ) {
        m_audioRead->slot_openAudio();
    }
}

```

```
}
```

2.3 数据添加到队列

```
void SaveVideoFileThread::slot_writeVideoFrameData(uint8_t *picture_buf, int
buffer_size)
{
    //记录第一次时间
    //投递队列时，每一帧数据需要 记录对应的时间
    if( m_videoBeginFlag ) {
        m_videoBeginTime = QDateTime::currentMSecsSinceEpoch();
        m_videoBeginFlag = false;
    }
    qint64 curTime = QDateTime::currentMSecsSinceEpoch();
    qint64 pts = curTime - m_videoBeginTime;
    videoDataQuene_Input( picture_buf , buffer_size , pts );

    //    以下为写入视频帧的函数
    //    int write_video_frame( ... );
    //    memcpy( video_st.frameBuffer , picture_buf , buffer_size );
    //    av_free( picture_buf );
    //    video_st.frame->pts = video_st.next_pts++;
    //    write_frame(oc, video_st.enc, video_st.st, video_st.frame);

}

void SaveVideoFileThread::slot_writeAudioFrameData(uint8_t *picture_buf, int
buffer_size)
{
    //视频如果没有，丢弃帧 拉齐视音频时间
    if( (m_avFormat.hasCamera || m_avFormat.hasDesk)&& m_videoBeginFlag )
    {
        free( picture_buf );
        return;
    }

    audioDataQuene_Input( picture_buf , buffer_size );

    //    以下为写入音频帧的过程
    //    int write_audio_frame( ... );
    //    AVCodecContext *c;
    //    c = audio_st.enc;

    //    memcpy( audio_st.frameBuffer , picture_buf , buffer_size );
    //    free( picture_buf );
    //    audio_st.frame->pts = audio_st.next_pts;
    //    audio_st.next_pts += audio_st.frame->nb_samples;
```

```

//    AVRational rational;
//    rational.num = 1;
//    rational.den = c->sample_rate;
//    audio_st.frame->pts = av_rescale_q(audio_st.samples_count, rational,
c->time_base);
//    audio_st.samples_count += audio_st.frame->nb_samples;
//    write_frame(oc, audio_st.enc, audio_st.st, audio_st.frame);
}

```

2.4 开启写数据线程

```

void SaveVideoFileThread::slot_setInfo(STRU_AV_FORMAT & avFormat)
{
/*
省略之前的代码，在最后添加一下代码
*/
    this->start();
}

```

2.5 写数据线程函数

```

void SaveVideoFileThread::run()
{
    while(1)
    {
        /* select the stream to encode */
        if ( have_video &&
            ( !have_audio || (av_compare_ts(video_st.next_pts, video_st.enc->time_base,
audio_st.next_pts, audio_st.enc->time_base) <= 0)) )
        {
            if (!write_video_frame(oc, &video_st, video_pts))
                msleep(1);
        }
        else
        {
            if (!write_audio_frame(oc, &audio_st))
                msleep(1);
        }

        if ( isStop )
        {
            if( ( have_audio && ( m_audioDataList.size() == 0 ) )
                || ( have_video && ( m_videoDataList.size() == 0 ) ) )

```

```

        {
            if( m_audioDataList.size() != 0 )
            {
                write_audio_frame(oc, &audio_st);
            }else if( m_videoDataList.size() != 0 )
            {
                write_video_frame(oc, &video_st, video_pts);
            }
        }
        if( m_audioDataList.size() == 0 && m_videoDataList.size() == 0)
            break;
    }
}

//回收的代码从 slot_closeVideo() 函数移植过来
/* Write the trailer, if any. The trailer must be written before you
 * close the CodecContexts open when you wrote the header; otherwise
 * av_write_trailer() may try to use memory that was freed on
 * av_codec_close(). */
av_write_trailer(oc);

/* Close each codec. */
if ( m_avFormat.hasCamera || m_avFormat.hasDesk )
    close_stream(oc, &video_st);
if ( m_avFormat.hasAudio)
    close_stream(oc, &audio_st);

if (!(fmt->flags & AVFMT_NOFILE))
    /* Close the output file. */
    avio_closep(&oc->pb);

/* free the stream */
avformat_free_context(oc);
}

```

2.6 写视频帧

根据原来的代码进行调整

```

bool SaveVideoFileThread::write_video_frame(AVFormatContext *oc, OutputStream *ost,
double time)
{
    qDebug() << __func__;
    /// 稳帧的处理, 首先根据下一帧的时间, 获取大于下一帧时间的数据
    /// 如果没有就播放上一画面
    BufferDataNode *node = videoDataQuene_get(time);
    if (node == NULL)
    {
        if( lastVideoNode )

```

```

        node = lastVideoNode ;
    else
        return false;
}
else
{
    if (node != lastVideoNode)
    {
        if (lastVideoNode != NULL)
        {
            av_free(lastVideoNode->buffer);
            free(lastVideoNode);
        }

        lastVideoNode = node;
    }
}
if( !node ) return false;

memcpy(ost->frameBuffer, node->buffer, node->bufferSize);
ost->frame->pts = ost->next_pts++;
return write_frame( oc , ost->enc , video_st.st , ost->frame , video_pts , ost);
}

```

```

int SaveVideoFileThread::write_frame(AVFormatContext *fmt_ctx, AVCodecContext
*c, AVStream *st, AVFrame *frame , int64_t & pts , OutputStream *ost )
{
    int ret;

    if( !frame ) return 0;

    // send the frame to the encoder
    ret = avcodec_send_frame(c, frame);
    if (ret < 0) {
        fprintf(stderr, "Error sending a frame to the encoder\n" );
        return 0 ;
    }

    while (ret >= 0) {
        AVPacket pkt = { 0 };

        ret = avcodec_receive_packet(c, &pkt);
        if (ret == AVERERROR(EAGAIN) || ret == AVERERROR_EOF)
            break;
        else if (ret < 0) {
            fprintf(stderr, "Error encoding a frame\n");
            return 0 ;
        }
    }
}

```

```

/* rescale output packet timestamp values from codec to stream timebase */
av_packet_rescale_ts(&pkt, c->time_base, st->time_base);
pkt.stream_index = st->index;

//todo 修改部分 下一帧数据时间戳 pts 单位 ms
pts = av_rescale_q( ost->next_pts, st->time_base, {1, 1000});

/* Write the compressed frame to the media file. */
ret = av_interleaved_write_frame(fmt_ctx, &pkt);
av_packet_unref(&pkt);

if (ret < 0) {
    fprintf(stderr, "Error while writing output packet\n");
    return 0 ;
}

return ret == AVERROR_EOF ? 1 : 0;
}

```

2.7 写音频帧

```

bool SaveVideoFileThread::write_audio_frame(AVFormatContext *oc, OutputStream *ost)
{
    qDebug() << __func__;
    AVCodecContext *c;
    c = audio_st.enc;

    //从队列里面获取
    BufferDataNode* node = NULL;
    node = audioDataQuene_get();
    if ( node )
    {
        memcpy(ost->frameBuffer, node->buffer, node->bufferSize);

        free(node->buffer);
        node->buffer = NULL;
        free(node);
        node = NULL;
    }
    else
    {
        return false;
    }
}

```



```

audio_st.frame->pts = audio_st.next_pts;
audio_st.next_pts += audio_st.frame->nb_samples;

AVRational rational;
rational.num = 1;
rational.den = c->sample_rate;
audio_st.frame->pts = av_rescale_q(audio_st.samples_count, rational, c->time_base);
audio_st.samples_count += audio_st.frame->nb_samples;

return write_frame(oc, audio_st.enc, audio_st.st, audio_st.frame, audio_pts, ost);
}

```

2.8 关闭视频录制

```

void SaveVideoFileThread::slot_closeVideo()
{
    if( m_avFormat.hasCamera || m_avFormat.hasDesk)
        m_picInPicRead->slot_closeVideo();

    if( m_avFormat.hasAudio )
        m_audioRead->slot_closeAudio();

    isStop = true;
}

```